Les langages de description d'architecture logicielle

- (ADL - *Architecture Description Languages*)

"L'architecture bien documentée est celle qui répond aux questions que les développeurs se poseront dans six mois, pas à celles que l'architecte avait en tête en la créant."

Simon Brown

Introduction

Il existe une panoplie de langages pour représenter une architecture logicielle, par exemple les langages naturels, UML (Unified Modeling Language) et les ADLs (Architecture Description Languages).

Introduction

- Les **langages de description d'architecture** (ADL - *Architecture Description Languages*) sont des outils formels ou semi-formel utilisés pour spécifier, documenter et analyser les structures logicielles.
- Contrairement à un langage de programmation qui décrit comment un système fonctionne, un ADL décrit ce qu'il est structurellement : ses composants, les interfaces qu'ils exposent, et les connecteurs qui les relient.
- Un ADL est un langage qui offre des fonctionnalités (une syntaxe et une sémantique formelle) pour la définition explicite et la modélisation de l'architecture conceptuelle d'un système logiciel.
- Ces langages permettent de capturer les aspects **statiques** (structure) et **dynamiques** (comportement, reconfigurations) d'un système.
- *"Un ADL est un langage qui fournit des abstractions pour modéliser l'architecture d'un système logiciel, permettant ainsi d'analyser, de raisonner et de communiquer à propos de cette architecture." — Medvidovic & Taylor, 2000*

Les concepts de base des ADLs

Bien que les ADL varient, la communauté s'accorde sur trois éléments clés constitutifs .

Quelle que soit leur approche, les ADL partagent un noyau de trois concepts fondamentaux :

- **Les composants,**
- **Les connecteurs,**
- **Les configurations.**

En plus de ces trois abstractions de base, certains ADL introduisent des concepts complémentaires :

- Les **ports** (points d'interaction d'un composant),
- Les **rôles** (points d'interaction d'un connecteur),
- Les **styles architecturaux** (patrons réutilisables de configuration),
- Les **propriétés non fonctionnelles** (performance, fiabilité, sécurité).

Les concepts de base des ADLs
Les composants

Les Composants : Unités de calcul ou de stockage dotées de **ports** pour communiquer avec l'environnement.

- **Nature et rôle :** Un composant est une unité logicielle autonome et cohérente qui fournit un ensemble de fonctionnalités. Il est souvent considéré comme une « boîte noire » dont l'implémentation interne est masquée.
- Pour communiquer avec son environnement, le composant utilise des points d'interaction explicites appelés **ports**. Ces ports exposent des **interfaces fournies** (services offerts) et des **interfaces requises** (services nécessaires)
- **Types de composants :** On distingue les composants **primitifs**, qui contiennent directement le code source, des composants **composites**, qui sont eux-mêmes constitués d'un assemblage de sous-composants

Les concepts de base des ADLs
Les connecteurs

- **Les Connecteurs :** Éléments de premier ordre modélisant les interactions et les protocoles entre composants.
- Les interfaces d'un connecteur sont appelées des **rôles**. Chaque rôle définit la manière dont un composant peut participer à l'interaction
- Les connecteurs encapsulent un **protocole d'interaction** complexe, gérant des aspects tels que l'ordre des messages, la synchronisation, la gestion des erreurs et les délais d'attente (*timeouts*)

Les concepts de base des ADLs
La Configuration

- **La Configuration :** Description de la manière dont les composants et les connecteurs sont reliés pour former le système complet.
- Une configuration est souvent représentée comme un **graphe annoté** où les nœuds sont les composants et les arcs sont les connecteurs.
- Le lien entre ces éléments est défini par des **attachements**. Un attachement associe formellement un **port** d'un composant à un **rôle** spécifique d'un connecteur
- La configuration impose des contraintes sémantiques sur l'assemblage, garantissant par exemple que seuls des ports et rôles compatibles peuvent être liés entre eux.

Rôles et avantages des ADL

L'utilisation d'un ADL offre plusieurs bénéfices majeurs :

- **Analyse et vérification :** Ils fournissent une base rigoureuse pour détecter les erreurs de conception dès les premières phases. Certains, comme **Wright**, permettent de vérifier la cohérence et la complétude des protocoles d'interaction.
- **Simulation et exécution :** Des langages comme **Rapide** permettent de simuler les conceptions architecturales pour en analyser les résultats.
- **Réduction de l'ambiguïté :** En imposant des modèles formels, ils limitent les incohérences internes et les interprétations erronées.
- **Génération de code :** Ils peuvent servir de plan directeur pour générer automatiquement des parties du système.

comparer les ADL

Pour comparer objectivement les ADL, on peut les évaluer selon les critères suivants :

Critère	Question posée
Expressivité	Le langage peut-il décrire tous les styles architecturaux courants ?
Formalisme	La sémantique est-elle précise et raisonnement formel possible ?
Analysabilité	Permet-il de vérifier des propriétés (deadlock, performance) ?
Outillage	Dispose-t-il d'outils de modélisation, simulation, génération de code ?
Interopérabilité	Peut-il échanger avec d'autres ADL ou notations ?
Adoption	Quelle est sa présence dans l'industrie et la recherche ?
Extensibilité	Peut-on ajouter de nouveaux types de composants/connecteurs ?

Description des principaux ADLs

- Les langages de description d'architecture (**ADL**) sont des notations formelles ou semi-formelles permettant de supporter la description textuelle de la **structure** et du **comportement** des systèmes logiciels.
- Il existe une grande diversité de langages, certains étant généralistes et d'autres spécifiques à un domaine: **Acme , Wright , Darwin , AADL** , **MetaH , C2 , UniCon**

Description des principaux ADLs
Acme

Acme (Architecture Configuration Management Environment) est un ADL généraliste développé à CMU[1] par David Garlan, Robert Monroe et Drew Kompanek.

Acme (Le langage d'échange)**:** Utilisé comme format d'échange commun.

L'une des caractéristiques les plus distinctives **d'Acme** est sa capacité à servir de **format d'échange commun** entre différents outils de conception architecturale.

Utiliser dans l'enseignement et la recherche pour expliquer les concepts de "styles architecturaux" et de "connecteurs", car sa syntaxe est beaucoup plus lisible que les standards industriels les plus récents.

1-UCM : Université Carnegie Mellon

Description des principaux ADLs

Acme

La syntaxe d'Acme repose sur plusieurs concepts:

Components (Composants)	• Les unités de calcul ou de stockage (ex: un serveur, une base de données).
Connectors (Connecteurs)	• Ils définissent comment les composants interagissent (ex: un appel RPC, un flux SQL, un bus de messages).
Ports	• Ce sont les points d'interaction d'un composant (ex: un port "requête" et un port "administration").
Roles (Rôles)	• Ce sont les interfaces d'un connecteur. Un connecteur "Client-Serveur" a deux rôles : le rôle "client" et le rôle "serveur".
Properties (Propriétés)	• Des annotations pour ajouter des informations supplémentaires (performances attendues, auteur, coût, etc.).
Systems (Systèmes)	• Configurations globales reliant composants et connecteurs.
Attachements (Attachments)	• Ils décrivent explicitement comment les ports des composants sont reliés aux rôles des connecteurs pour former le système.

Description des principaux ADLs

Acme

Rep-Maps

- Une des forces d'Acme est sa capacité à gérer des niveaux de détails différents.
- Lorsqu'un composant est décrit par une **Représentation** (un sous-système interne), Acme utilise des **Rep-Maps** (Representation Maps).
- Elles servent de "pont" entre les ports du composant externe et les ports des composants internes. Cela garantit que l'interaction qui arrive de l'extérieur est correctement routée vers le bon élément à l'intérieur.

Styles (ou Familles)

- Acme permet de définir des **styles architecturaux** (familles d'architectures), ce qui permet aux concepteurs de spécialiser le vocabulaire et les contraintes pour des types spécifiques de systèmes (par exemple, un style *Pipe-and-Filter)*
- Un style est un ensemble de règles que les systèmes doivent suivre. Cela permet de créer des modèles cohérents et d'éviter les erreurs de conception dès le départ.

Armani

- Pour rendre Acme encore plus intelligent, on lui associe souvent **Armani**, un langage de contraintes.
- Armani permet d'écrire des **règles logiques** du type : *"Un serveur ne peut pas être connecté à plus de 50 clients", "Tous les composants de type 'Sécurisé' doivent être reliés par des connecteurs 'Chiffrés'."*
- Si l'architecte dessine un système qui viole l'une de ces règles, Acme le signalera immédiatement.

Description des principaux ADLs
Acme

- Le langage est supporté par **AcmeStudio** (CMU), un environnement d'édition personnalisable et un outil de visualisation. Cet outil permet aux architectes de définir de nouvelles familles Acme et de personnaliser l'interface graphique pour travailler avec ces familles en définissant des styles de diagrammes spécifiques.

- Acme est privilégié pour sa **flexibilité**, permettant à la fois de décrire des structures statiques rigoureuses et de servir de pont entre différents outils de modélisation grâce à sa nature de langage d'échange

Description des principaux ADLs
Acme

Description d'une architecture client serveur simple dans ACME

```
System simple_cs = {
Component Client = { Port send-request }
Component Serveur = { Port receive-request }
Connector rpc = { Roles {caller, callee} }
Attachments = {
client.send-request to rpc.caller ;
server.receive-request to rpc.callee }
}
```

Description des principaux ADLs

Acme

simple repository system

```
System simple_repository_system = {
Component client = {
        Port request;
        Property avg_trans_per_sec: int;
 }
Component db = {
        Port provide;
}
Connector db_access = {
        Role user;
        Role provider;
}
Attachments = {
        client.request to db_access.user ;
        db.provide to db_access.provider;
 }
 }
```

Le client est relié à la base de données par un connecteur d'accès nommé « db_access ».

Description des principaux ADLs
Wright

Wright est un ADL formel développé à l'Université Carnegie Mellon (CMU) par Robert Allen et David Garlan au milieu des années 1990.

Wright se distingue par son approche formelle issus de (CSP - *Communicating Sequential Processes*), centrée sur la spécification et l'analyse des interactions entre les composants logicielles

Il est considéré comme l'un des premiers ADL à formaliser rigoureusement la notion de **connecteur** comme entité architecturale de première classe, dotée d'une sémantique propre et non réductible à un simple lien entre composants.

Wright utilise les mêmes concepts de base (Composants, Connecteurs), mais il y ajoute une couche de **comportement** (Behavior).

Il permet de **décrire le comportement dynamique** des systèmes, pas seulement leur structure.

Wright a été conçu pour détecter les **Deadlocks** avant même d'écrire le code.

On l'utilise quand on ne peut pas se permettre une erreur de logique de communication (systèmes de contrôle aérien, centrales nucléaires, bancaire, dispositifs médicaux).

Description des principaux ADLs
Wright

Les 3 piliers de Wright

Composants

- Dans Wright, un composant est défini par un ensemble de **ports**.
- Chaque **port** décrit le comportement observable du composant à ce point d'interaction, exprimé en termes de processus CSP (Communicating Sequential Processes).
- Le composant dispose également d'un calcul (computation) qui décrit son comportement interne.

Connecteurs

- **C'est la contribution majeure de Wright**.
- Un connecteur est une entité de première classe possédant des **rôles** (l'équivalent des ports pour les connecteurs) et une **glue** (colle) qui spécifie comment les rôles s'articulent.
- Cette distinction permet de modéliser des protocoles d'interaction complexes : appel de procédure, pipe, événement, rendez-vous, etc.

Configurations

- Une configuration assemble composants et connecteurs en un système.
- Elle déclare les instances, leurs types, et les attachements (liaisons entre ports de composants et rôles de connecteurs).

Description des principaux ADLs
Rapide

- **Rapide** est un ADL développé à l'Université de Stanford par David Luckham et James Vera dans les années 1990.
- Son originalité est de décrire l'architecture logicielle en termes de flux d'événements et de contraintes sur ces flux.
- Rapide permet non seulement de spécifier une architecture, mais aussi de la simuler et d'en vérifier le comportement par rapport à des contraintes temporelles et causales.
- Dans Rapide, chaque composant possède une interface qui définit les événements qu'il peut émettre (out) et recevoir (in). Les événements sont les unités fondamentales de communication — Rapide ne connaît pas d'appels de procédure directs.
- Le comportement d'un composant est décrit par des règles d'exécution : si l'événement E1 est reçu, alors émettre E2. Les contraintes (poset constraints) spécifient des relations de causalité et de précédence entre événements. Ces contraintes permettent de définir les propriétés que l'architecture doit satisfaire.
- Les connexions Rapide définissent comment les événements émis par un composant sont acheminés vers d'autres composants. Une connexion peut être simple (un-à-un), broadcastée (un-à-plusieurs), ou filtrée (conditionnelle).
- **La capacité de simulation est le point fort distinctif de Rapide.** À partir d'une description architecturale et d'un ensemble d'événements d'entrée, Rapide génère une simulation d'exécution (un poset d'événements) qui peut être comparée aux contraintes spécifiées.

Description des principaux ADLs
Darwin

Darwin : Adapté à la description des structures logicielles distribuées et des topologies de réseau.

Darwin est un ADL développé à l'Imperial College de Londres par Jeff Magee et Jeff Kramer. Sa particularité essentielle est de traiter l'architecture comme une entité dynamique.

Darwin permet de décrire non seulement la structure statique initiale d'un système, mais aussi ses reconfigurations au cours de l'exécution — création de nouveaux composants, suppression d'instances, modification de liaisons.

Darwin supporte la description d'architectures statiques avec des composants primitifs (implémentés en Java ou C) et des composants composites qui en assemblent d'autres.

La puissance de Darwin réside dans la description de comportements dynamiques.

Darwin permet de spécifier des règles de reconfiguration : instanciation conditionnelle de composants, suppression d'instances, redirection de liaisons en cours d'exécution.

Description des principaux ADLs
UniCon

- **UniCon :** Propose un compilateur de haut niveau pour des designs supportant des types de composants et connecteurs hétérogènes.
- UniCon (Universal Connector) est un ADL développé à CMU par Mary Shaw et ses collègues. Il adopte une approche centrée sur les connecteurs et les styles architecturaux.
- UniCon fournit un catalogue de types de connecteurs prédéfinis (pipes, procédures, événements, données partagées...) que le concepteur assemble pour définir des configurations architecturales.
- UniCon distingue les composants (players) et les connecteurs (connectors). Chaque connecteur prédéfini a des protocoles précis qui régissent comment les composants peuvent l'utiliser.

Description des principaux ADLs
UniCon

Les principaux types de connecteurs UniCon sont :

Pipe	Flux de données unidirectionnel, FIFO
Procedure-call	Appel synchrone avec passage de paramètres
Event	Diffusion asynchrone d'événements
Data-access	Accès à des données partagées
File-System	Accès via chemins de fichiers
Remote-procedure	Appel de procédure à distance

- UniCon est l'un des premiers ADL à formaliser la notion de style architectural : un style est un patron récurrent de composants et connecteurs, avec des contraintes sur leur assemblage.
- UniCon supporte nativement les styles : pipeline (pipes-and-filters), client-serveur, tableau noir (blackboard), appel-retour (main-program/subroutine) et architecture en couches.

Description des principaux ADLs AADL

AADL (*Architecture Analysis and Design Language*) : Standard pour l'ingénierie prédictive des systèmes temps réel et embarqués (avionique, automobile).

- **Année de création : 2004 (v1), 2009 (v2)**
- **Version actuelle : AADL v3 (2023)**
- **AADL** est un standard **SAE International**, initialement développé par **SEI (Software Engineering Institute)** de Carnegie Mellon.
- Il est spécialisé dans la modélisation des **systèmes embarqués temps-réel** critiques: avionique, automobile, spatial et médical
- il permet avec analyse formelle des propriétés non-fonctionnelles.

Description des principaux ADLs AADL

AADL Components

core modeling elements
- *Components*
- *Interactions*
- *Properties*

Application Software

- thread
- thread group
- process
- data
- subprogram

thread
thread group
process
data
Subprogram

Execution Platform

- processor
- memory
- bus
- device

processor
memory
bus
device

Composite

- system

System

Each component has predefined properties associated with its declaration.

Description des principaux ADLs AADL

- AADL distingue rigoureusement les composants logiciels, matériels et composites. Cette séparation est essentielle pour analyser le déploiement et l'adéquation logiciel/matériel :

Catégorie	Type	Description
Logiciel	Data	Données statiques (variables, structures)
	Subprogram	Unité de calcul appelable
	Thread	Unité d'exécution concurrente
	Thread Group	Groupement logique de threads
	Process	Espace d'adressage contenant des threads
Matériel	Processor	Unité de traitement physique
	Memory	Mémoire physique
	Bus	Canal de communication physique
	Device	Périphérique d'entrée/sortie
Composite	System	Assemblage de composants logiciels et matériels

Description des principaux ADLs AADL

Pourquoi l'AADL est différent des autres ?

L'AADL résout le problème du "canal" entre le logiciel et le matériel que les anciens ADLs ignoraient :

- **Dualité Logiciel/Matériel :** Contrairement aux ADL qui ne décrivent que du code, l'AADL permet de modéliser les composants logiciels (Process, Thread, Data) **ET** les composants matériels (Processor, Memory, Bus, Device).
- **La notion de "Binding" :** On peut spécifier précisément quel thread logiciel s'exécute sur quel processeur physique. (crucial pour calculer la latence réelle d'un système de freinage ou d'un pilote automatique.)
- **Analyse Prédictive :** L'AADL n'est pas juste un dessin. Sa syntaxe est assez stricte pour permettre à des outils (comme OSATE) de prouver mathématiquement que le système ne ratera jamais une échéance temporelle (*deadline*).

Description des principaux ADLs AADL

- La force d'AADL est d'associer des propriétés quantitatives à chaque composant, permettant une analyse formelle des contraintes temps réel. Ces propriétés incluent notamment :
  - Période et échéance des threads (Period, Deadline)
  - Temps d'exécution pire cas (Worst-Case Execution Time — WCET)
  - Priorité et politique d'ordonnancement (Scheduling_Protocol)
  - Latence des bus et bande passante
  - Espace mémoire et processeur

- AADL bénéficie d'un écosystème outillage industriel mature :
  - OSATE (Open Source AADL Tool Environment) — IDE Eclipse pour modéliser et analyser des architectures AADL
  - Ocarina — compilateur AADL générant du code Ada/SPARK ou C pour systèmes embarqués
  - Cheddar — outil d'analyse d'ordonnançabilité temps réel
  - Stood — outil commercial de modélisation AADL (Ellidiss Technologies)

Description des principaux ADLs AADL

Exemple : système d'alerte de collision dans une voiture connectée
Dans un **ADL classique** (comme ACME), on dirait juste : "Le Capteur envoie une donnée au Contrôleur".
En **AADL**, on descend dans le détail "physique" pour garantir que le freinage s'activera à temps.

```
-- 1. Définition du composant logiciel (Thread)
thread Detection_Collision
features
  distance_entree : in data port;
  alerte_sortie : out data port;
properties
  Dispatch_Protocol => Periodic;
  Period => 20ms; -- Le calcul doit se faire toutes les 20ms
end Detection_Collision;

-- 2. Définition du composant matériel (Processeur)
processor Intel_Automotive
properties
  Scheduling_Protocol => (Posix_1003_1b);
end Intel_Automotive;
```

Description des principaux ADLs UML

- UML (Unified Modeling Language) est un standard de l'OMG (Object Management Group) publié en 1997. Bien que généraliste (il couvre l'ensemble du cycle de développement logiciel),
- UML est le langage de modélisation le plus utilisé dans l'industrie pour décrire les architectures logicielles.
- UML 2.x propose 14 types de diagrammes, dont plusieurs sont particulièrement pertinents pour la description architecturale :

Diagramme	Usage architectural
Diagramme de composants	Décomposition en composants et interfaces
Diagramme de déploiement	Mapping logiciel/matériel, nœuds de déploiement
Diagramme de paquetages	Organisation modulaire du code source
Diagramme de classes	Structure des entités et leurs relations
Diagramme de séquences	Interactions dynamiques entre composants
Diagramme d'activité	Flux de traitement, workflows
Diagramme de cas d'utilisation	Fonctionnalités du point de vue de l'utilisateur

Description des principaux ADLs UML

Limites d'UML pour la description architecturale

- Malgré son adoption universelle, UML présente des limitations reconnues pour la description architecturale :
- Connecteurs sans sémantique : UML ne formalise pas les protocoles d'interaction (contrairement à Wright). Un connecteur est juste un trait entre deux boîtes.
- Multiplicité des diagrammes : une architecture complète nécessite plusieurs diagrammes hétérogènes maintenus en cohérence manuellement.
- Sémantique formelle incomplète : la spécification UML laisse des ambiguïtés d'interprétation, rendant la vérification formelle difficile.
- Verbosité graphique : les diagrammes UML complexes deviennent rapidement illisibles.

ADL vs UML

- Bien que l'**UML** (*Unified Modeling Language*) soit le langage le plus utilisé par les praticiens pour modéliser l'architecture, une distinction est souvent faite:
  - **UML** est souvent considéré comme une **notation graphique** qui nécessite des conventions (stéréotypes) pour représenter certains concepts architecturaux comme les couches.
  - Un ADL est **formel**, ce qui permet de faire des analyses mathématiques.
  - Un véritable **ADL** possède une **sémantique opérationnelle** rigoureuse qui le rend exécutable ou interprétable par une machine abstraite.
  - Certains ADLs peuvent détecter automatiquement si deux composants ne peuvent pas se parler (incompatibilité de protocoles) ou bien prouver que le système ne va pas "geler" (deadlock) avant même d'avoir écrit une seule ligne de code.

Description des principaux ADLs SysML

- SysML (Systems Modeling Language) est un profil UML standardisé par l'OMG et l'INCOSE (International Council on Systems Engineering) en 2006. Il étend UML pour couvrir les besoins de l'ingénierie des systèmes (Systems Engineering).
- SysML modélise tout type de système —logiciel, matériel, données, personnes et processus via des blocs et des flux, pas seulement des logiciels orientés objet.
- **Année de création :** 2006 (v1.0)
- **Version actuelle : SysML v2.0 (2024)**
- Le concept central de SysML est le bloc (Block)
- Block: une unité de modélisation plus riche qu'une classe UML.
- Un bloc peut avoir des **flow properties** (propriétés de flux) pour modéliser **des échanges d'énergie**, de matière ou de données, ainsi que **des value properties** (propriétés valeurs) pour **les mesures quantitatives**.

Description des principaux ADLs SysML

- SysML remplace certains diagrammes UML et en introduit de nouveaux .
- NASA, ESA, Airbus utilisent SysML pour modéliser les systèmes embarqués de satellites et d'avions.
- **Papyrus** (Eclipse) — Outil open source, académique

Diagramme SysML	Équivalent UML
Block Definition Diagram (BDD)	Diagramme de classes
Internal Block Diagram (IBD)	Diagramme de structure composite
Parametric Diagram	Aucun
Requirements Diagram	Aucun
Activity Diagram (ACT)	Diagramme d'activité UML
Sequence Diagram	Diagramme de séquence UML
State Machine Diagram (STM)	Machine à états UML
Use Case Diagram	Diagramme de cas d'utilisation UML
Package Diagram	Diagramme de Package

Description des principaux ADLs C4

- Le modèle C4 (Context, Container, Component, Code) a été créé par Simon Brown vers 2011 comme réponse pragmatique aux limitations d'UML pour la documentation architecturale dans les équipes agiles.
- Il ne s'agit pas d'un ADL formel au sens académique, mais d'une notation légère et accessible, organisée en quatre niveaux de zoom progressifs.

Niveau	Nom	Contenu	Audience	Granularité
1	Context (Contexte)	Système + utilisateurs + systèmes externes	Direction, métier, tous	Très faible
2	Container (Conteneur)	Applications, services, bases de données, jobs	Architectes, tech leads	Faible
3	Component (Composant)	Modules, packages, classes principales d'un conteneur	Développeurs	Moyenne
4	Code	Classes, interfaces, fonctions	Développeurs	Élevée

Description des principaux ADLs C4

- C4 définit un vocabulaire visuel simple et précis :
  - Personne : un utilisateur humain du système (client, administrateur, opérateur)
  - Système logiciel : l'application ou le service considéré, ou les systèmes externes
  - Conteneur : une unité déployable séparément (application web, API, base de données, worker)
  - Composant : un groupement de fonctionnalités au sein d'un conteneur
  - Relation : une dépendance entre éléments, avec une description de la nature de l'interaction
- **Structurizr** est l'outil officiel de Simon Brown pour créer des diagrammes C4.

Son DSL (Domain Specific Language) textuel permet de décrire l'architecture sous forme de code versionnable, générant automatiquement les quatre niveaux de diagrammes C4.

Description des principaux ADLs ArchiMate

- ArchiMate est un langage de modélisation d'architecture d'entreprise développé par The Open Group en 2002 et standardisé en 2009.
- ArchiMate est un **langage de modélisation visuel** développé par The Open Group (standard depuis 2009. Il est conçu pour décrire l'architecture d'entreprise dans sa globalité : stratégie, métier, applications, technologie, physique.

- **Version actuelle : ArchiMate 3.2 (2022)**
- ArchiMate est le langage de référence pour les architectes d'entreprise, souvent associé au framework TOGAF.
- ArchiMate est le langage graphique recommandé pour documenter les livrables du framework TOGAF (The Open Group Architecture Framework). Il fournit une notation standardisée là où TOGAF est un cadre de processus.

Description des principaux ADLs ArchiMate

- Le modèle fondamental d'ArchiMate organise l'architecture en 6 couches horizontales, chacune répondant à un niveau d'abstraction différent:

Couche	Contenu	Éléments typiques
Métier (Business)	Processus, acteurs, services métier, contrats	Business Process, Business Role, Business Service
Application	Applications, composants, services applicatifs, données	Application Component, Application Service, Data Object
Technologie	Infrastructure matérielle et logicielle système	Node, Device, System Software, Network, Communication Path

ArchiMate 3 ajoute des aspects supplémentaires : la couche Stratégie (capabilities, ressources), la couche Physique (équipements physiques, matériaux) et des éléments de motivation (drivers, goals, requirements).

Description des principaux ADLs ArchiMate

ArchiMate définit un vocabulaire précis de relations entre éléments :

Relation	Signification
Composition	Un élément est composé d'autres (relation partie-tout)
Agrégation	Un élément agrège d'autres sans les posséder
Assignment	Un acteur ou rôle est assigné à un élément comportemental
Realization	Un élément réalise concrètement un élément logique
Serving	Un élément offre ses services à un autre
Triggering	Un événement déclenche un autre comportement
Flow	Flux d'information ou de ressource entre éléments
Association	Relation générale entre éléments

Description des principaux ADLs ArchiMate

- ArchiMate est conçu pour fonctionner en synergie avec TOGAF (The Open Group Architecture Framework).
- TOGAF fournit le processus (ADM — Architecture Development Method) et ArchiMate fournit la notation pour documenter les livrables à chaque étape du processus. Les quatre domaines d'architecture TOGAF (Business, Data, Application, Technology) correspondent aux couches et aspects ArchiMate.
  - Stratégie : Capability · Resource · Course of Action · Value Stream
  - Métier (Business) : Actor · Role · Business Process · Business Service · Business Object
  - Application: Application Component · Application Service · Application Function · Data Object
  - Technologie : Node · Device · System Software · Technology Service · Artifact Physique Facility · Equipment · Distribution Network · Materiel

Guide de choix d'un ADL

Aujourd'hui, l'utilisation des ADLs "purs" (comme Wright ou Darwin) reste très académique ou réservée à des domaines ultra-critiques.

Les langages de description d'architecture (ADL) constituent un spectre large allant des formalismes mathématiques rigoureux (Wright, Rapide, AADL) aux notations pragmatiques accessibles à tous (C4, ArchiMate). Aucun ADL n'est universellement optimal : chacun répond à un contexte, des contraintes et une audience spécifiques.

Plusieurs tendances se dessinent pour l'avenir des ADL :

- L'architecture as code (Structurizr DSL, PlantUML, Mermaid) intègre la documentation architecturale dans les pipelines CI/CD, garantissant qu'elle reste synchronisée avec le code.
- Le glissement vers le pragmatisme : les équipes agiles privilégient les notations légères (C4) aux ADL formels trop lourds à maintenir.
- La convergence modèle/code : des outils comme OSATE (AADL) génèrent du code embarqué directement depuis le modèle, réduisant l'écart entre spécification et implémentation.
- L'essor de SysML v2 : la version 2 de SysML introduit une sémantique formelle et un format d'échange standardisé (KerML), marquant une rupture avec la v1 basée sur UML.

Guide de choix d'un ADL

Contexte	ADL recommandé	Justification
Système embarqué avionique/automobile	AADL	Standard industriel, analyse temps réel, outillage mature
Architecture d'entreprise / TOGAF	ArchiMate	Standard Open Group, trois couches métier/appli/tech
Équipe agile, documentation rapide	Modèle C4 + Structurizr	Accessible, versionnable, génération automatique
Système cyber-physique complexe	SysML	Intègre matériel, logiciel, flux et exigences
Recherche, vérification formelle	Wright ou Rapide	Sémantique formelle, vérification de propriétés
Interopérabilité entre outils ADL	Acme	Langage pivot, extensible
Architecture distribuée dynamique	Darwin	Supporte la reconfiguration à l'exécution
Modélisation généraliste en entreprise	UML 2.x	Standard universel, outillage abondant

L'aspect statique dans les ADLs.

- L'**aspect statique** d'un ADL décrit la structure d'un système telle qu'elle existe *avant* et *indépendamment* de son exécution : quels composants existent, quelles interfaces ils exposent, et comment ils son connectés.
- L'aspect statique se déploie sur trois niveaux emboîtés : les **types** forment le vocabulaire réutilisable, les **instances** peuplent une architecture concrète, et la **configuration** assemble le tout en une topologie.
- L'aspect statique d'un ADL répond à une seule question :

  quelle est la structure du système, indépendamment du temps ?

L'aspect statique dans les ADLs.

Les éléments statiques fondamentaux:

- **Composants**

Un composant est l'unité de traitement ou de stockage du système. Sa description statique comprend toujours au moins deux choses : son **type** (ce qu'il est) et ses **ports** (comment il se connecte au reste). Selon les ADL, on y ajoute des propriétés non fonctionnelles, des représentations internes, ou des contraintes.

  - Dans **Wright**, un composant déclare ses ports avec leur comportement CSP (Communicating Sequential Processes) et un calcul interne
  - Dans **Acme**, la syntaxe est plus légère mais extensible par propriétés
  - Dans **AADL**, un composant est fortement typé (Thread, Process, Processor…) et porte des propriétés quantitatives

- **Connecteurs**

Le connecteur est la contribution théorique majeure des ADL par rapport à UML. Dans la plupart des notations graphiques, un connecteur est un simple trait. Dans un ADL, c'est une **entité de première classe** avec son propre type, ses rôles et sa sémantique d'interaction.

un connecteur UML n'est qu'un trait sans sémantique propre.

L'aspect statique dans les ADLs.

Les éléments statiques fondamentaux:

- **Ports et rôles**

Les ports et rôles sont les **points d'interaction** :

- les ports appartiennent aux composants,
- les rôles appartiennent aux connecteurs.

La distinction est essentielle : un port dit *ce qu'un composant offre ou requiert* ; un rôle dit *ce qu'un connecteur attend de ses participants*.

L'attachement (port ↔ rôle) est la colle structurelle de la configuration.

- **Propriétés statiques**

Les propriétés sont des annotations non fonctionnelles associées à tout élément de la description statique. Elles permettent de capturer des attributs de qualité sans changer la structure.

Une **configuration** est la description statique d'un système complet : elle liste les instances de composants et de connecteurs, et déclare tous les attachements qui les relient. C'est le niveau où l'architecture devient un graphe topologique précis.

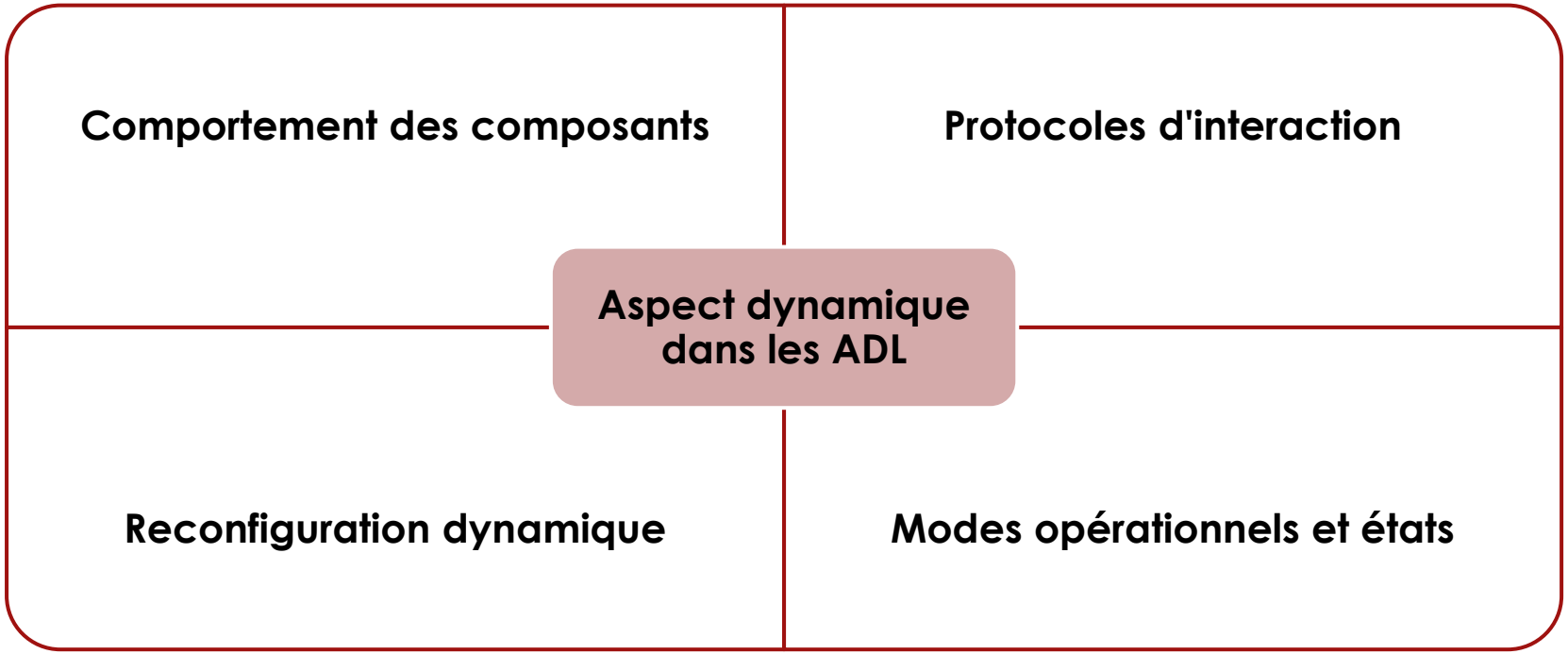
L'aspect statique dans les ADLs.

Les styles architecturaux — contraintes statiques globales

- Un **style architectural** est un ensemble de contraintes statiques qui s'appliquent à une famille de configurations. Il définit quels types de composants et connecteurs sont autorisés, quelles topologies sont valides, et quelles propriétés doivent être respectées. C'est la notion la plus abstraite de l'aspect statique.
- Contraintes statiques communes à tout style architectural :
  - Types de composants autorisés
  - Types de connecteurs autorisés
  - Topologies valides / interdites
  - Propriétés obligatoires
  - Invariants structurels

L'aspect dynamique dans les ADL

- Si l'aspect statique décrit *ce qu'est* un système, l'aspect dynamique décrit *ce qu'il fait au cours du temps* : comment les composants se comportent, comment ils interagissent, et — pour les ADL les plus avancés — comment la structure elle-même peut **changer pendant l'exécution**.
- L'aspect dynamique couvre quatre dimensions que les ADL adressent avec des degrés de richesse très différents.

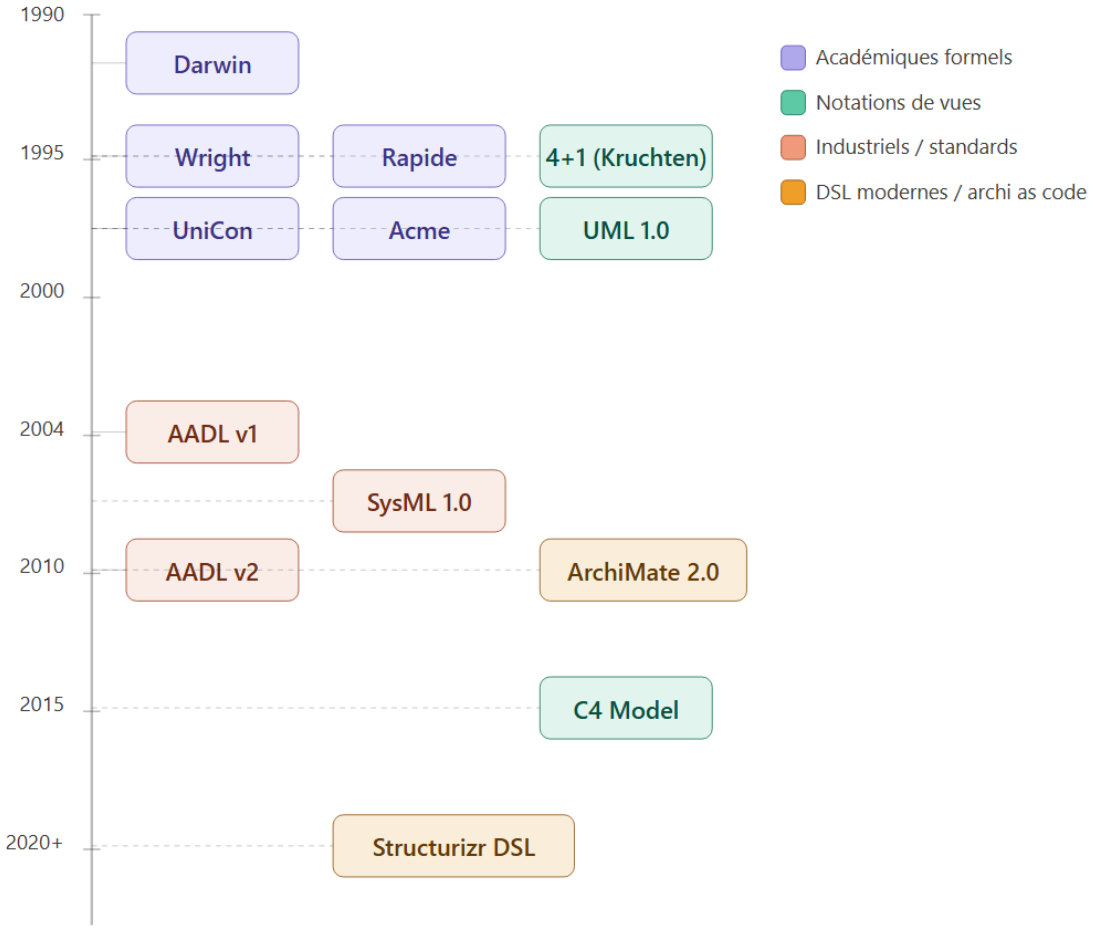


Aspect dynamique dans les ADL

- **Comportement des composants :** concerne le **comportement observable** d'un composant individuel. qu'est-ce qu'il fait, dans quel ordre, sous quelles conditions ?
- **Protocoles d'interaction :** C'est la dimension dynamique la plus originale des ADL. Un connecteur n'est pas juste un fil : il encapsule un **protocole d'interaction** entre ses participants, qui se déroule dans le temps.
- **Reconfiguration dynamique :** C'est la dimension la plus avancée, et celle qui distingue fondamentalement Darwin des autres ADL. La **reconfiguration dynamique** signifie que la structure du système elle-même peut changer pendant l'exécution : de nouveaux composants sont créés, des instances sont supprimées, des liaisons sont redirigées.
- **Modes opérationnels et machines à états :** Plusieurs ADL modélisent le comportement dynamique d'un système comme une **machine à états** : le système passe d'un mode à un autre selon des événements ou des conditions, et chaque mode définit une configuration active différente.

Évolution Chronologique des ADLs

L'évolution suit une transition nette : on passe de langages académiques rigides à des langages industriels plus flexibles et orientés vers le "Cloud Native"



Classification des ADLs

On distingue quatre grandes familles d'ADL :

1. **ADL académiques formels** (Wright, Rapide, Darwin, Acme, UniCon) — Nés dans les laboratoires de recherche dans les années 1990, ils mettent l'accent sur une **sémantique formelle rigoureuse** — composants, connecteurs, configurations — permettant la vérification et le raisonnement formel sur l'architecture. Peu adoptés industriellement, mais fondateurs conceptuellement.

Chacun a apporté une contribution spécifique :

- **Acme** — langage d'échange et d'interopérabilité entre ADL
- **Wright** — formalisation des interactions via les algèbres de processus (CSP)
- **Darwin** — modélisation des systèmes distribués dynamiques
- **Rapide** — simulation et analyse du comportement temporel
- **UniCon** — unification de styles architecturaux hétérogènes

2. **Notations de vues** (C4, 4+1) —Des cadres **pragmatiques et non formels** qui organisent la documentation architecturale en plusieurs perspectives complémentaires (logique, processus, déploiement, scénarios…). L'objectif est la communication entre parties prenantes plutôt que la vérification formelle.

Classification des ADLs

3. **ADL industriels et standards** (UML, SysML, AADL) Conçus pour une **adoption à grande échelle**, ils bénéficient d'un outillage commercial mature (modélisateurs, générateurs de code, simulateurs). AADL est particulièrement présent dans les systèmes embarqués critiques (aérospatial, défense).
   - **SysML** — profil UML pour la modélisation de systèmes complexes (systèmes embarqués, ingénierie système)
   - **AADL** — standard SAE dédié aux systèmes embarqués temps-réel critiques (aérospatial, automobile, défense)

4. **DSL modernes et architecture as code** (ArchiMate, Structurizr DSL) —Des approches **contemporaines** qui intègrent la description d'architecture (la documentation) directement dans le flux de développement — versionnée, automatisée, synchronisée avec le code. Elles répondent aux besoins des équipes agiles et DevOps.
   - **ArchiMate** — langage de modélisation pour l'architecture d'entreprise (standard The Open Group)
   - **Structurizr DSL** — description textuelle et versionnée d'architectures logicielles, basée sur le modèle C4

(DSL - **Domain-Specific Language** (langage dédié à un domaine spécifique)

Vérification et validation

- La **vérification** et la **validation** servent à garantir qu'une architecture logicielle est à la fois **correcte techniquement** et **adaptée aux besoins**. Sans elles, un ADL ne serait qu'un simple outil de dessin.

- La **vérification (Building the system right)** répond à la question "Est-ce que le système est bien construit?" (conformité à la spécification),

- La **validation (Building the right system)** répond à "Est-ce que c'est le bon système ?" (adéquation aux besoins réels).

- Dans le contexte des ADL, ces deux activités s'appliquent à la description architecturale elle-même — avant même qu'une ligne de code soit écrite.

- La vérification (**cohérence interne**) et la validation (**adéquation au besoin**) sont deux faces complémentaires de la qualité architecturale.

- Contrairement à des notations purement graphiques, les ADL **reposent souvent sur des bases formelles** qui permettent des analyses rigoureuses

Vérification

Objectif : Détecter des **erreurs structurelles ou comportementales**.

Vérification structurelle	Assurer que l'architecture est bien formée :	ports correctement connectés
		types compatibles
		respect des styles
Vérification comportementale	Vérifier les interactions dynamiques :	ordre des messages correct
		synchronisation respectée
		absence de conflits

Propriétés vérifiées

- ☐ Deadlock (blocage)
- ☐ Livelock (activité inutile infinie)
- ✔ Cohérence des échanges
- ✔ Complétude des interactions

Techniques utilisées

- Model checking
- Analyse formelle
- Simulation de comportements

Validation dans les ADLs

Objectif : S'assurer que l'architecture répond aux **besoins réels**.

Validation par scénarios

- On teste des cas d'utilisation

Validation des exigences

- Vérifier que l'architecture respecte : performance, sécurité, fiabilité

Validation par simulation

- Exécuter le modèle pour observer le comportement

Validation par styles architecturaux

- Vérifier que le choix d'architecture est adapté

Vérification et validation

ATAM (Architecture Tradeoff Analysis Method, SEI) est la méthode de validation architecturale la plus reconnue. Elle réunit architectes, développeurs et représentants métier pour évaluer l'architecture par rapport à des attributs de qualité concrets (performance, disponibilité, modifiabilité, sécurité).

Le processus se déroule en 8 phases.

Les phases 0 à 3 constituent l'investigation : présentation de la méthode, recueil des drivers métier (objectifs stratégiques, contraintes), présentation de l'architecture par l'architecte, identification des approches architecturales utilisées.

Les phases 4 à 7 constituent l'analyse proprement dite : construction d'un arbre d'utilité (hiérarchie de scenarios de qualité priorisés H/M/L), analyse des approches par rapport aux scenarios, brainstorm de nouveaux scenarios avec les parties prenantes, analyse approfondie des scenarios prioritaires. La phase 8 restitue les résultats.

Les livrables d'ATAM sont : les risques (points potentiellement défaillants), les non-risques (décisions solides), les points de sensibilité (décisions architecturales dont un léger changement affecte fortement un attribut de qualité), et les compromis (décisions affectant positivement un attribut et négativement un autre).

Vérification et validation

SAAM (Software Architecture Analysis Method) est le prédécesseur d'ATAM, plus simple, centré sur les scenarios de changement — convient mieux aux architectures en phase de conception initiale.

Fitness functions — validation continue dans le pipeline

La notion de **fitness function** (Ford & Parsons, *Building Evolutionary Architectures*, 2017) est la réponse moderne à la validation architecturale dans les contextes agiles et DevOps : des tests automatisés, exécutés à chaque build, qui mesurent en continu si l'architecture reste conforme à ses objectifs.

Vérification et validation

ADL	Technique principale	Propriétés vérifiables	Outil	Automatisation
Wright	Model checking CSP	Deadlock, conformité interface	FDR2	Complète
Rapide	Simulation de posets	Causalité, contraintes temporelles	Simulateur Rapide	Semi-auto
AADL	Analyse temps réel	Ordonnançabilité, WCET, latence	OSATE, Cheddar	Complète
UML	Model checking + revue	Deadlock, états, séquences	SPIN, PAT	Partielle
Acme	Vérification de types	Cohérence structurelle	AcmeStudio	Complète
C4 + code	Fitness functions	Dépendances, cycles, couplage	ArchUnit, Structurizr	Complète (CI)
ATAM	Revue par scénarios	Attributs de qualité	Manuel + ateliers	Non

Vérification et validation

L'intégration de la V&V dans les ADL offre des bénéfices concrets pour le cycle de vie du logiciel :

- **Détection précoce des fautes :** En validant la logique avant même le début du codage, on réduit les coûts de correction.
- **Sûreté de fonctionnement :** Pour les systèmes critiques (avionique, spatial), des langages comme **AADL** permettent de prédire les propriétés de performance et de fiabilité.
- **Conformité du code :** Certains outils (comme ArchJava) utilisent des systèmes de types pour garantir que l'implémentation finale respecte strictement les contraintes architecturales (intégrité de communication)

La vérification et la validation dans les ADL transforment l'architecture d'un simple document passif en un **outil de conception prédictif** capable de garantir mathématiquement la viabilité du système